

# Lifelong Learning for a Kresling-Origami Robot: A Plug-and-play Module for Supervised Learning of Target Adaptations

E. De Vroey

**Abstract—...**

## I. INTRODUCTION

## II. BACKGROUND

### A. Soft Robotics

challenges

### B. Lifelong Learning

parameter adaptation

### C. Kresling Origami

usage in soft-robotics

bistable behavior

design parameters

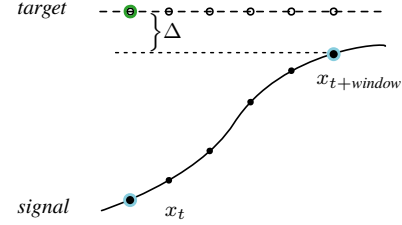
## III. METHODOLOGY

### A. Target Adaptations

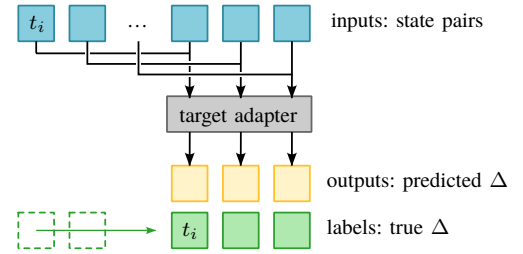
The method proposed in this paper aims to provide a way of improving a given controller's performance as well as fortifying it against evolving system dynamics *without* having access to the inner workings of the controller itself. In other words, as long as the fundamental assumptions of a controller are satisfied—i.e. it requires a state and a target as input and yields a control action—the method can remain completely agnostic to the specific controller being used. In addition, the aim is not to replace the controller entirely but to introduce an additional module to the control scheme that enables lifelong robustness to evolving system dynamics. The advantage of such a method is that it could be applied to any system as a *plug-and-play* module.

The manner in which this will be achieved is by *target adaptation*. An *adapter* module modifies the target used by the controller to improve its performance. Consider a scenario where a controller yields a control action  $u$  to reach a certain desired target state  $\mathbf{x}_d$ . As the system dynamics evolve, suppose this action  $u$  is now insufficient to reach the target. In response, the adapter module *lies* to the controller about reaching  $\mathbf{x}_d$  and instead provides it with  $\mathbf{x}_d + \Delta$ , for which the controller yields a larger action  $u^*$  that *does* result in the system reaching state  $\mathbf{x}_d$ .

The *adapter* module is implemented as a feedforward neural network with to hidden layers with non-linear activations. To



(a) The target adaptation problem: Based on the current position  $x_t$  and the position some prediction window later  $x_{t+window}$ , the model learns to predict the target the controller is trying to reach by representing it as a  $\Delta$  to be added to  $x_{t+window}$ .



(b) The training process of the target adapter: recorded states spaced by a specified prediction window are paired up and provided as features to the adapter, which learns to predict a  $\Delta$ , being the difference between the target  $\mathbf{x}_d$  and state  $\mathbf{x}_{t+window}$ .

Fig. 1: Target adaptation during the training phase

train this model to learn suitable target adaptations, interactions of the controller with the system are recorded. The model is then provided with pairs of system states, spaced by a certain time window  $\mathbf{x}_t$  and  $\mathbf{x}_{t+window}$ , that are obtained from these recordings. From these pairs, the model infers the target it ‘believes’ the controller aims to reach. However, instead of predicting the target directly, the model learns the difference  $\Delta$  between the target and the latter state in the pair (Figures 1a and 1b).

The model can be trained in a supervised manner (Figure 1b) and then deployed. During deployment, it is presented with the current state  $\mathbf{x}_t$  but cannot be presented with  $\mathbf{x}_{t+window}$  (since this state is not known yet). Instead, the second state in the pair is set to the desired target. To the model, this represents a situation in which the desired target was reached from the current state within the specified time window. The idea is now that the trained model will produce a  $\Delta$  that, when added to the latter state in the pair (i.e., the desired target), results in an adapted target (Figure 2a) for which the controller will produce a control action. In short, given a current state  $\mathbf{x}_t$  and a future state  $\mathbf{x}_{t+window}$ , the model learned to produce the desired target.

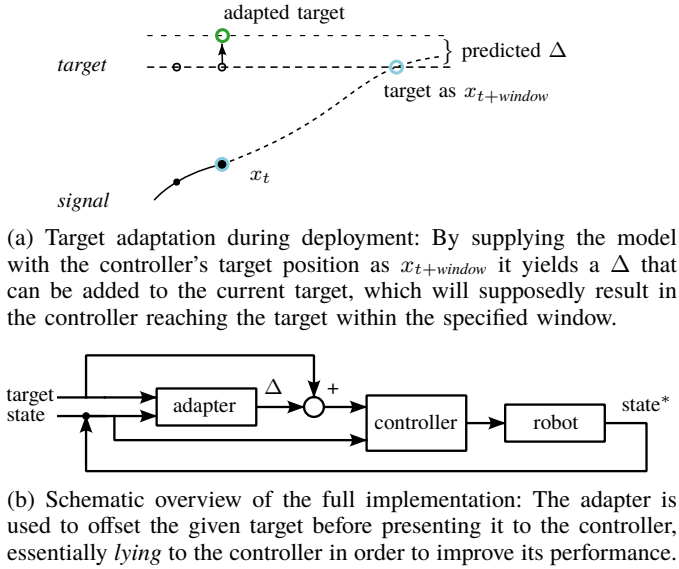


Fig. 2: Target adaptation during deployment.

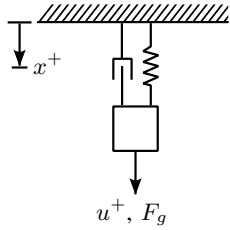


Fig. 3: Setup of the mass-spring-damper system in the toy problem with positive directions indicated.

Thus, when presented with  $\mathbf{x}_t$  and  $\mathbf{x}_{t+window} = \mathbf{x}_d$  (supposedly reached within the time window) it provides the (adapted) target that would “push” the controller toward that result. A full overview is presented in Figures 2a and 2b.

The reason for predicting the difference  $\Delta$  between the state  $\mathbf{x}_{t+window}$  and the desired target, rather than a direct prediction of the target, can also be clarified now: By designing the model's output to be the difference  $\Delta$ , the model can be initialized with parameters all close to zero (essentially leaving the desired target unchanged) and then steadily improve online as interactions of the controller and system are recorded, without the need for a pretraining phase.

### B. Toy Problem

To explore the proposed method, a toy problem is considered first. The toy problem consists of a simulation of a simple 1D mass-spring-damper system (Figure 3) and an imperfectly tuned PID controller. The mass is suspended in a hanging position with gravity acting on it continuously.

The system state  $\mathbf{x}$  consists of the position and velocity  $x, \dot{x}$ , respectively. The controller aims to reach targets of the form  $\mathbf{x}_d = [x_d, 0]$ , but controls position only. At regular intervals, a random target position  $x_d$  is generated for the controller to reach and maintain. Each timestep, the adapter calculates the required  $\Delta$ , and the controller computes the control action

based on the current position and the adapted target position. The system's response to this control input is simulated and the tuple  $(x_0, \dot{x}_0, u, x, \dot{x}, x_d + \Delta, 0)$  is added to the buffer. This buffer stores a limited number of tuples corresponding to a specified window of the most recent data. Once capacity is reached, the oldest element is removed at every timestep. At regular update intervals, the adapter is trained on the data in the buffer. It is important that the tuples in the training data always contain the target position that was actually provided to the controller, i.e.,  $x_d + \Delta$ , and not the *true* desired target  $x_d$ . As the simulation progresses, the parameters of the dynamic system are gradually altered to simulate system degradation. The pseudo algorithm for the implementation can be found in Algorithm 1.

---

#### Algorithm 1: Toy problem target adaptation

---

**Data:** system state  $\mathbf{x}$ , target  $\mathbf{x}_d$ , adaption  $\Delta$ , control action  $u$

Initialize model: *adapter*;

Initialize buffer:  $buffer \leftarrow []$ ;

**for** each timestep  $t_i$  **do**

**if**  $len(buffer) > threshold$  **then**

Remove the oldest element in the buffer;

**end**

**if**  $t_i \bmod target\_interval = 0$  **then**

$\mathbf{x}_d \leftarrow random\_target()$ ;

**end**

**if**  $t_i \bmod update\_interval = 0$  **then**

$(features, labels) \leftarrow prep\_data(buffer, window)$ ;

$adapter.fit(features, labels)$ ;

**end**

$\Delta \leftarrow adapter.predict([\mathbf{x}_0, \mathbf{x}_d])$ ;

$u \leftarrow controller.compute\_control(\mathbf{x}_0, \mathbf{x}_d + [\Delta, 0])$ ;

Simulate the system response  $\mathbf{x}$ ;

Update system parameters;

Append  $(\mathbf{x}_0, u, \mathbf{x}, \mathbf{x}_d + [\Delta, 0])$  to  $buffer$ ;

$\mathbf{x}_0 \leftarrow \mathbf{x}$ ;

**end**

---

### C. Preliminary Results

The toy problem was implemented in Python: simulating the system response using SciPy's `lsim`, and implementing the adapter in Keras with the PyTorch backend. The simulation ran for 60s at 50Hz. The target was changed every 5s, and the adapter was trained fully online, updating every 15s. Training features were created for multiple prediction windows, this means the training features consisted of pairs  $[\mathbf{x}_t, \mathbf{x}_{t+window}]$  for windows of 3, 5, 10, 15, and 25. The buffer's budget corresponded to the past 30s of data. For reference, a second controller and system were also simulated, identical to the described simulation, but without the addition of an adapter. To both the reference and adapted signal, a small amount of Gaussian noise was added to simulate measurement noise.

1) *A case where the controller is too aggressive:* For a first simulation, a combination of a system and controller was chosen where the controller's gains were tuned too high.

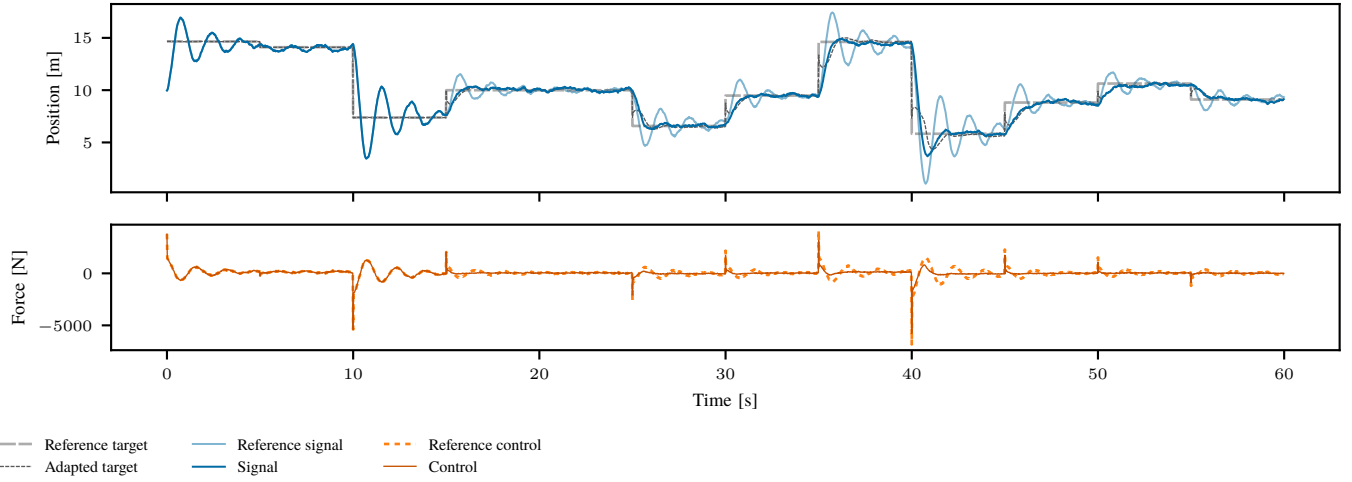


Fig. 4: The full minute simulation of the toy problem with an aggressive controller. Top: The system response  $x$  and target  $x_d + \Delta$  of the adapted controller, alongside the response and target of the unadapted reference controller ( $\Delta = 0$ ). Bottom: The resulting adapted control inputs from the controller receiving  $x_d + \Delta$  alongside the unaltered reference control inputs. During the first 15s (3 targets) the adapter has not received updates yet, and the signal is thus identical to the reference signal.

In Figure 4, the result of the simulation can be observed. At the start, and due to the near-zero initialization of the adapter's parameters, the signal response of the system controlled by the adapted controller is indistinguishable from that of the reference controller. However, as soon as the adapter has trained on the buffer, the changes to the target become visible.

The effects of the adaptation can be more clearly observed in Figure 5. Since the reference controller is too aggressive, the reference signal overshoots and oscillates significantly before settling. In contrast, the adapter counters this by bringing the target closer to the current state. This is the logical result of the buffer containing data corresponding to the overshoots, where the adapter would often encounter training features for which the labeled target position<sup>1</sup>  $x_d$  is somewhere in between  $x_t$  and  $x_{t+window}$ . Therefore, upon receiving the input feature  $[\mathbf{x}_t, \mathbf{x}_d]$  it produces an adapted target position in between  $x_t$  and  $x_d$ . As the state approaches the adapted target, the adapter starts producing targets closer to the reference target, guiding the system towards this desired state.

2) *A case where the controller is under-responsive:* For a second simulation, the system and controller were designed such that the controller was more *conservative* or *under-responsive*. Additionally, the signal produced by the reference controller has a clear steady-state error. The result of the simulation can be found in Figure 6.

Once more, the adapter is able to improve the controller's performance after just a single update. In Figure 7 it can be seen that the adapted target is adjusted away from the current state. This is because training features created from the data in the buffer frequently include state pairs  $\mathbf{x}_t, \mathbf{x}_{t+window}$  where the target position  $x_d$  is positioned further from  $x_t$  than it is from  $x_{t+window}$ . As a result, the adapter learns to predict

<sup>1</sup>Rather, the labeled  $\Delta = x_d - x_{t+window}$ , but the explanation provided is equivalent and somewhat clearer.

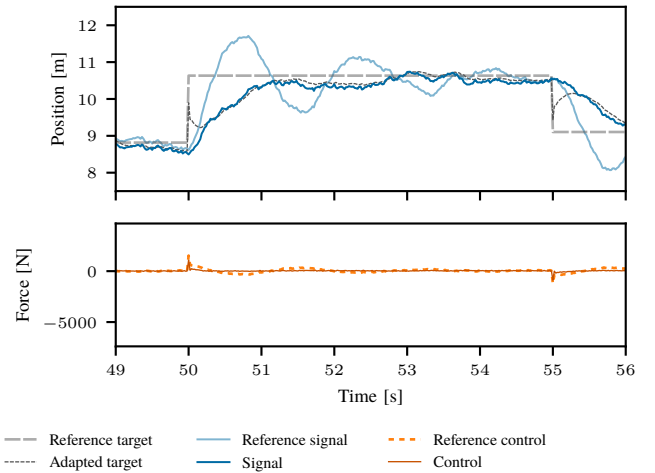


Fig. 5: A close-up of the second to last target in the one-minute simulation. The effects of the adapter can be more clearly observed: The overshoot (seen in the reference signal) is countered by bringing the target closer to the current state.

adaptations that would move  $x_d$  farther from  $x_t$ , making the controller more aggressive. Another effect of the adapter is the consistent offset applied to the target: In the training features, the adapter regularly encounters pairs  $\mathbf{x}_t, \mathbf{x}_{t+window} = \mathbf{x}_t$ , which appear to represent the target state being reached, even though  $x_{t+window} \neq x_d$ . These features are the result of the steady-state error present in the signal, leading the adapter to learn to apply an offset to the target to counteract this.

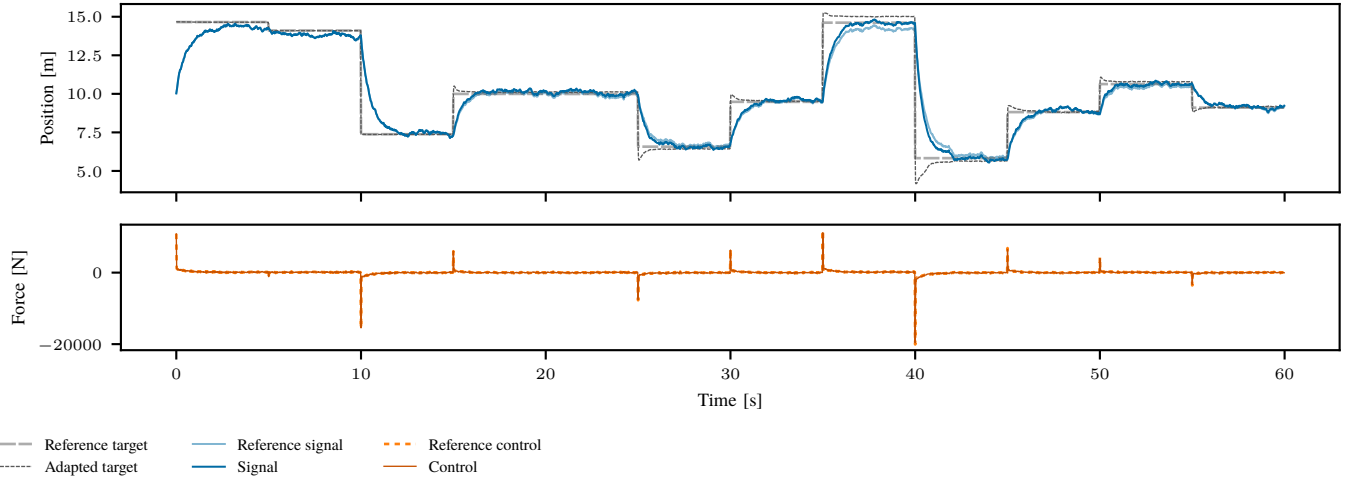


Fig. 6: The full minute simulation of the system with an under-responsive controller. Top: The system response and target of the adapted controller, alongside the response and target of the unadapted reference controller. Bottom: The resulting adapted control inputs from the controller receiving the altered target alongside the unaltered reference control inputs.

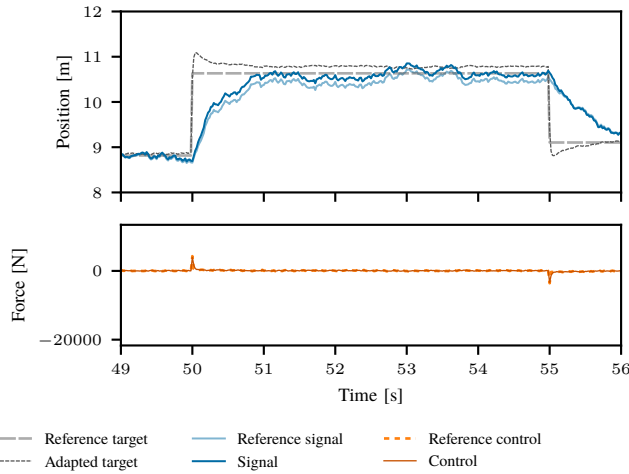


Fig. 7: A close-up of the second to last target in the one-minute simulation. Notable observations: The target is initially moved further from the current state to ‘pull’ it towards the desired state in a shorter period. In addition, the adapted target is offset from the desired state as to counter the steady-state error seen in the reference signal.

#### IV. A KRESLING ORIGAMI ROBOT

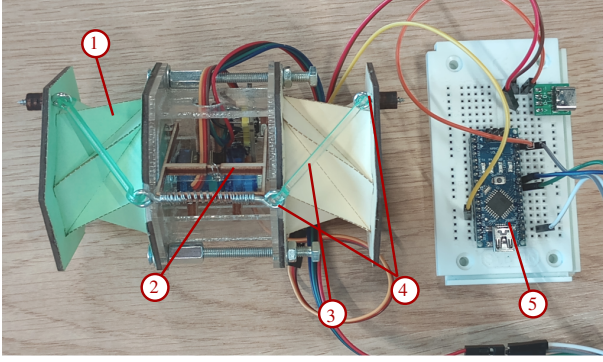
##### A. Mechanical Details

As mentioned in Section II, origami robots—and by extension, soft robots—are a suitable use-case for lifelong learning techniques. To evaluate the method described in Section III, a robot was designed that serves as a good analogue to the toy problem, whilst also using kresling origami as a key part of its functionality. Figure 8 provides important nomenclature for reference.

The robot is a simple crawler consisting of two Kresling origami springs (KOS) (Figure 8a, item 1) that are mirror images of each other, connected in series, with the actuation mechanism in between. Both springs can expand and contract in sync, producing a crawling-like motion when combined with specially designed “feet”. To demonstrate target adaptation, however, the movement of the robot as a whole is not very important. Rather, the state of the KOSs is the state that will be controlled.

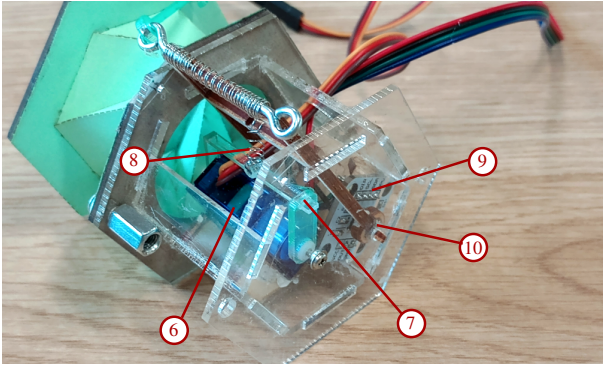
A KOS can be actuated both by linear motion (pushing or pulling on the spring) and by rotational motion (twisting). Since the two motions are directly related (Section II-C), a contracting or expanding motion can be measured as the angle of the KOS’ twist. The twist is induced by turning a rotating arm (2) which is connected to the ends of the KOSs by two links (3) that hook into eye-screws (4). To turn the arm, a micro servo motor (6) is employed. In the toy problem, the controller yields a force to exert on the system in order to reach the target position. However, most micro servo’s come with a near perfect position controller built-in, which cannot be accessed or changed. Therefore, to bridge the gap between the simulation and the real-world implementation, the position output of the servo is converted to a force output by the use of a small, metal spring (8). The servo thus does not directly act on the actuation arm, but rotates a servo arm (7) instead, which pulls the spring and thereby exerts a force on the actuation arm. As previously mentioned, the state of the KOS can be measured as the angle of twist, since it is directly related to its contraction. Likewise, assuming perfectly rigid links with negligible clearance between the hooks and eye-screws, the angle of twist is directly related to the angle of the actuation arm. In the full implementation, the robot’s state  $\mathbf{x}_t$  will thus be measured as the actuation arm’s angular position and velocity  $[\theta, \dot{\theta}]$ . This angle is measured using a contactless sensor (9) that can measure the on-axis rotation





- |                    |              |
|--------------------|--------------|
| 1) KOS             | 4) Eye screw |
| 2) Actuation arm   | 5) Arduino   |
| 3) Connecting link |              |

(a) The Kresling origami robot.



- |                 |                 |
|-----------------|-----------------|
| 6) Servo motor  | 9) Angle sensor |
| 7) Servo arm    | 10) Magnet      |
| 8) Metal spring |                 |

(b) Actuation mechanism.

Fig. 8: Annotated views of the robot and its most important actuation mechanisms

of a small magnet (10) attached to the actuation arm. The sensor relays this information to an Arduino Nano (5) board, which runs a PID controller that returns the control action, i.e., an angular position for the servo. Furthermore, there are mechanical limits for the rotation of both the actuation arm and servo arm, imposed by the edge of the container housing the actuation mechanism and the supports holding the servo motor, respectively (Figure 9, right). However, the design is such that the servo's mechanical limit corresponds to a  $\theta$  below the actuation arm's limit.

### B. KOS characteristics

section on the KOS specifics

### C. Software Implementation

The supervised learning implementation for target adaptation is divided into two key components: the nominal control of the robot, programmed in C++ and executed by the Arduino board, and the target adaptation, implemented in Python and managed by an external computer.

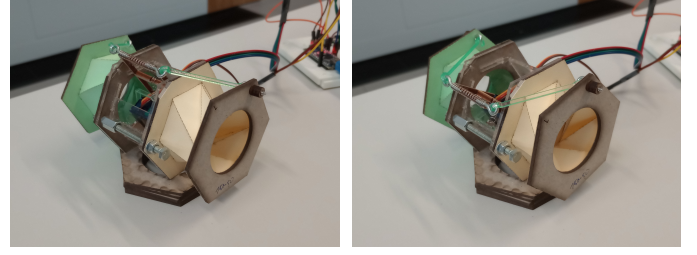


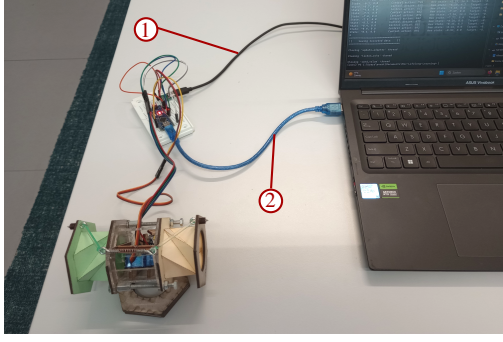
Fig. 9: *Left*: The robot at its fully extended state, corresponding to the “home” position of the servo and actuation arm. *Right*: The fully contracted robot, corresponding to the maximum (capped) servo output, near the mechanical limit of the actuation arm.

1) *Nominal Control on the Arduino*: Upon startup, the Arduino initializes a connection with the external computer and directs the servo to move to its minimum position. The Arduino allows some time for the servo to reach this home position and then reads a first measurement from the angle sensor, setting this as the reference  $\theta = 0$  (see Figure 9, left). During the control loop, the servo reads  $\theta$  (relative to the reference) and  $\dot{\theta}$  received from the sensor, and reads the target  $\theta_d$  received from the external computer. Based on these measurements, the PID controller computes the value to be written to the servo, but before writing it to the servo, a ceiling is imposed on it to protect the servo from exceeding its mechanical limit (Section IV-A). The Arduino allows some time for the servo to act and then reads the measurements from the angle sensor again. Finally, it sends the tuple containing the state, action, resulting state, and target to the external computer.

2) *Target Adaptation on the External Computer*: The target adaptation and supervised learning are implemented exclusively on the external computer through the established connection with the Arduino. The program is divided into several modules or *threads* that run simultaneously. This way, values that need to be sent or read can be updated in the background upon request, without delaying the operations of other modules (see Appendix A for a more detailed breakdown of the interactions between various threads). At regular pre-defined intervals, a `change_value` thread changes the true target  $\theta_d$  to some value. Every iteration, it constructs an input feature  $[\theta, \dot{\theta}, \theta_d, 0]$  for the adapter. The resulting  $\Delta$  is added to the target, which is read by the `send` thread and forwarded to the Arduino. The buffer is maintained by storing the tuples received from the Arduino. Upon train-time, the adapter weights are updated in the background, without interrupting the adapter's performance.

## V. EXPERIMENTAL SETUP

For each experiment, a minimum of two sets of KOSs was required: One set serving as a reference, with the Arduino receiving unadapted targets, and the second set used for testing target adaptation. All experiments used a 60s buffer and 15s intervals between updates, with the first update at 60s. The adapter trained on prediction windows of 3, 5, 10, 15, and 25



- 1) Servo motor power supply
- 2) Serial connection and Arduino power supply

Fig. 10: The test setup.

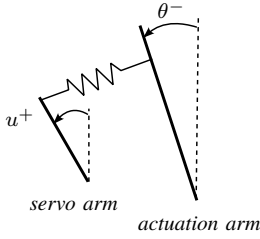


Fig. 11: Nominal signs of the angles for the servo and actuation arm during operation.

steps. The true target, with values depending on the experiment, was updated every 5s, following a predefined random sequence. This sequence amounts to 5min of deployment time and is repeated according to the full experiment duration. The signals corresponding to each 5min sequence are recorded and saved for evaluation. This way, the evolution of the performance metrics can be studied, and a fair comparison between the reference controller and adapted controller is ensured. Figure 10 shows the general experimental setup.

#### A. Sign Conventions and Unit Clarification

Before the experiments and results can be properly understood, it is necessary to clarify certain sign and unit conventions. Most importantly, the contactless magnetic sensor denotes a clockwise rotation (as viewed from the top) w.r.t. the datum as a positive angle, and likewise, a counterclockwise rotation as a decrement in the angle. In practice, this means that a positive control input from the servo (i.e. a positive angle) results in a decrement in  $\theta$ . Because the actuation arm's calibration point is set when the servo is at its minimum output, the operational range of the actuation arm is entirely in the negative domain. Figure 11 illustrates the servo arm and actuation arm in a configuration representative of a typical scenario during the experiments, with the signs of the angles indicated.

In addition, The *Servo* Arduino library uses the `writeMicroseconds(value)` function to write to the servo, which maps a servo's range to a value between 1000 and 2000 $\mu$ s depending on the servo's manufacturer. As such, `writeMicroseconds` provides more 'resolution' than the

common `write(angle)`, which works with integer values of angles in degrees. Note that the term *microseconds* in this context does not refer in any way to the angular unit of measurement *arcseconds*. Instead, it relates to the pulse width in microseconds of the signal that the Arduino sends to the servo motor. In this case, the servo's minimum position corresponds to a value of 540 $\mu$ s and the maximum mechanical limit (as discussed in Section IV-A) corresponds to approximately 1800 $\mu$ s. It might thus be confusing to see the control input, the angle of the servo arm, denoted in a unit of time. For convenience, the 540 $\mu$ s value relates to 0 degrees, and the 1800 $\mu$ s limit corresponds to 125 degrees.

Finally, during the experiments, the Arduino initialized a *counter* value to include in the tuple sent to the computer. This counter starts at 0 and simply increments by 1 with each iteration of the Arduino's loop. Since the interactions between the Arduino and computer, as well as between the several threads on the computer, are independent of timing (i.e., the modules do not have to "wait for each other"), the counter's value is essential to accurately plot the several signals recorded during the experiments. As a result, the counter itself does not measure elapsed time directly, making it unitless. However, it can be used as a proxy for time, since all modules aim to operate at 50Hz. This means that an increment of 1 in the counter corresponds to an estimated 0.02s, allowing for the approximation of elapsed time based on the counter value.

#### B. Performance Metrics

The performance metrics used to evaluate the 5min sequences are the mean rise time, mean settling time, mean overshoot, mean absolute error, integrated absolute error, mean absolute velocity, normalized total variance, and mean control action.

The mean rise time is calculated as the mean of all the rise times corresponding to each target in the 5min sequence. The rise time is defined as the time it takes the robot to come within 10% of the target state, as calculated from the initial position.

Similarly, the settling time is the average of the settling times for each target. The robot state is considered settled when it remains within 5% of the target state, as calculated from the initial position.

The mean overshoot is the average of the maximum overshoots observed for each target.

The mean absolute error represents the average difference between the robot's angle  $\theta$  and the true target  $\theta_d$ .

The mean absolute velocity refers to the measurements of  $\dot{\theta}$ . This metric is meant to provide a measure of the "chaoticness" of the robot's signal response.

The normalized total variance is calculated as the sum of the absolute of all increments and decrements in the control action, normalized over time<sup>2</sup>. This aims to quantify the controller's "effort".

<sup>2</sup>Actually, it is normalized over the *count*, see Section V-A.

## VI. EXPERIMENTS AND RESULTS

The performance of the target adapter is evaluated over three types of experiments. In the first type, the PID controller's tuning is varied, while maintaining consistent KOS characteristics. For the second group of experiments, the KOS characteristics are varied, namely the degree of perforation in the folds, while keeping the PID controller identical across tests. Finally, the performance is tested for a crawling gait to observe the effect of the method in a highly restricted setting.

### A. Varying the PID Controller

For the first experiment, the method's robustness to the use of different PID controllers is tested. The robot is equipped with the standard set of KOSs, using perforations of 1.5–0.5mm cut-skip ratio. During this experiment, the values of the targets in the random sequence are integer values ranging between -3 and -25 degrees. The robot has 5s to reach and settle around a target before it is provided with the next target in the sequence.

1) *A controller that is slow but aggressive:* The first PID controller has a disproportionally high integral gain, resulting in *aggressive* but *slow to respond* behavior. This controller yields particularly interesting results in combination with the bistable nature of the KOS. Figure 12a shows the signal response from the reference controller, illustrating the behavior of the controller. In comparison, the signal obtained from the controller-adapter combination is shown in Figure 12b.

During the full 2h duration of the experiment, the evaluation of each 5min sequence was recorded. The result can be seen in Figure 13.

Note the immediate (after the first 5min) improvement in overshoot and absolute error. The controller with target adapter initially has high rise times and low overshoots, but then evolves toward a stable behavior that retains short rise time, small position errors, and consistent overshoot values. As time progresses, the metric values for the unadaptive reference controller generally decrease, with the exception of the rise time. The adapted controller, however, seems to achieve this level of performance from the start.

*Interpretation:* The conjecture is that the potential barrier between the two stable states of the KOS slows down the controller's response, resulting in a higher control input. As the barrier is overcome, the KOS now complements the controller's action, resulting in the overshoot. Depending on the vicinity of the target to the KOS's potential barrier, the controller can enter an unstable state. As the potential barrier to transition between the two stable states of the KOS decreases, the reference controller has less difficulty maintaining a stable response. This explains the steady decrease in settling time, absolute velocity, and total variance, which reflect the decreasing irregularities in the signal and the controller's corrective actions. The target adapter effectively shifts the target closer to the system's current state, facilitating a smoother transition to the desired state with reduced overshoot, thereby minimizing the risk of the controller entering an unstable cycle.

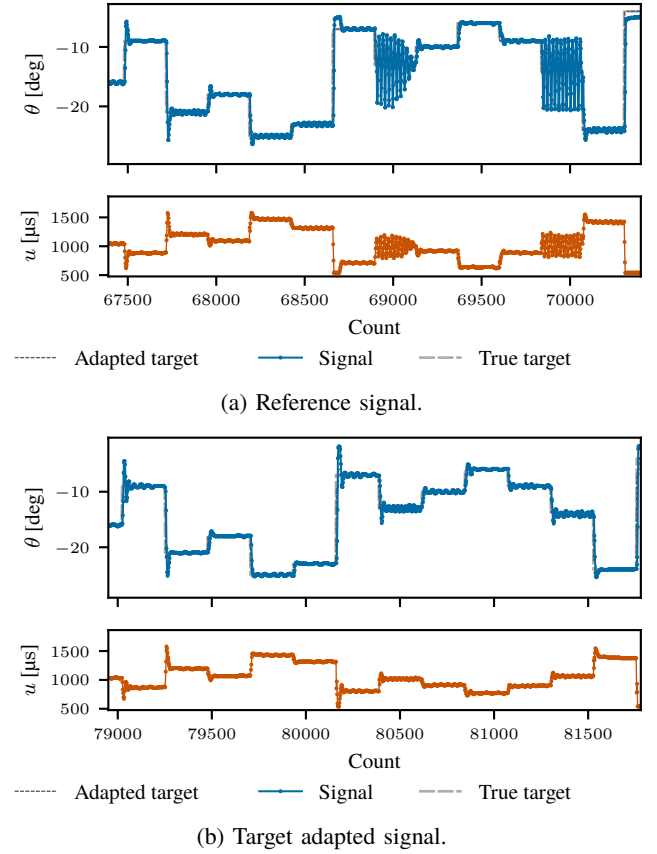


Fig. 12: Qualitative comparison of 1min of the signal response for the *slow-aggressive* PID controller.

2) *An under-tuned controller:* The second PID controller is tuned with low gains, resulting in slow and under-responsive behavior. Figures 14a and 14b display the comparison between the reference controller and the controller with target adapter, respectively.

The evolution of the metrics is shown in Figure 15.

The controller with adapter clearly has lower rise times and absolute errors than the reference. However, it also exhibits higher overshoots, whereas the reference controller even slightly undershoots toward the end of the 2h experiment. The adapted controller results in higher mean absolute velocity and total variance.

*Interpretation:* The signal in Figure 14b shows a clear reason for the lower rise times, namely that the target is shifted to more extreme positions as to yield larger control inputs from the controller. This corresponds to the observed larger overshoot and higher mean absolute velocity. The adapter makes the controller more *aggressive*, as seen in the increased controller *effort* (total variance). However, the adapter also seems to introduce unnecessary adaptations once the steady state is reached, further increasing the mean absolute velocity. These small oscillations are insignificant compared to the overall improvement in the mean absolute error.

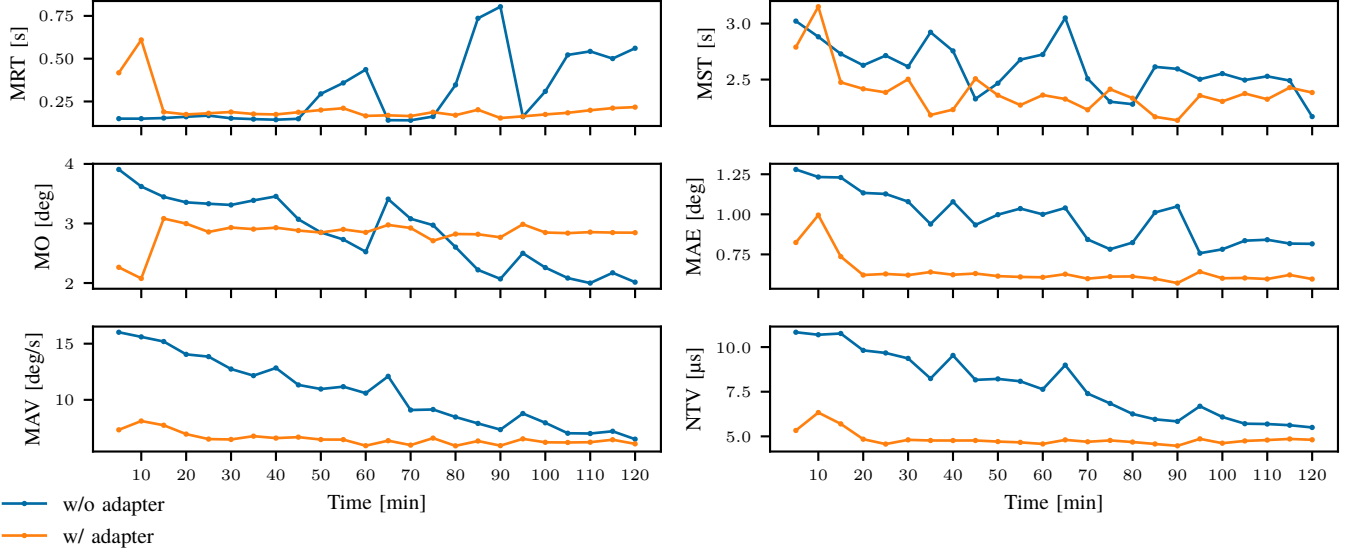


Fig. 13: Metrics evolution over the 2h duration of the experiment for an *aggressive* but *under-responsive* PID controller. Each point represents the calculated value of the metric on identical 5min sequences of targets. Numerical values can be found in Appendix C.

### B. Varying the Perforation

For the second type of experiment, the PID controller is tuned to a highly responsive performance on a robot with KOS perforations of a 1.5–0.5 mm skip-to-cut ratio. The KOS perforations used are 0.25–0.25 mm, 0.5–0.5 mm, 1.5–0.5 mm, and 2–0.5 mm, representing various stages of KOS degradation. Once again, the values of the targets in the random sequence are integer values ranging between -3 and -25 degrees, and the robot has 5s to reach and settle around a target.

1) *0.25–0.25 & 0.5–0.5 Perforations:* The 0.25–0.25mm and 0.5–0.5mm perforations represent KOSs in a state of lower degradation than the standard 1.5–0.5mm perforation. The evolution of the metrics over the course of the 1h experiment is shown in Figure 16, showing the average of the results from both experiments.

On average, the adapted controller has lower rise times, settling times, and absolute errors, but higher overshoot, absolute velocity, and total variance. Another observation is that the difference in the reference controller's performance between the two experiments drifts further apart over the full duration. However, the adapted controller's performance over the two experiments appears less varied. Overall, the relative difference in metrics between reference and adapted controller resembles that of the under-tuned PID experiment.

*Interpretation:* Since the perforations aim to represent a less degraded KOS, i.e. stiffer, the gains of the PID (tuned to a 1.5–0.5mm perforated KOS) are likely set too low. The situation is thus analogous to the under-tuned PID experiment, and this shows in the behaviour of the target adapter: The targets are shifted toward more extreme positions, resulting in the lower rise time.

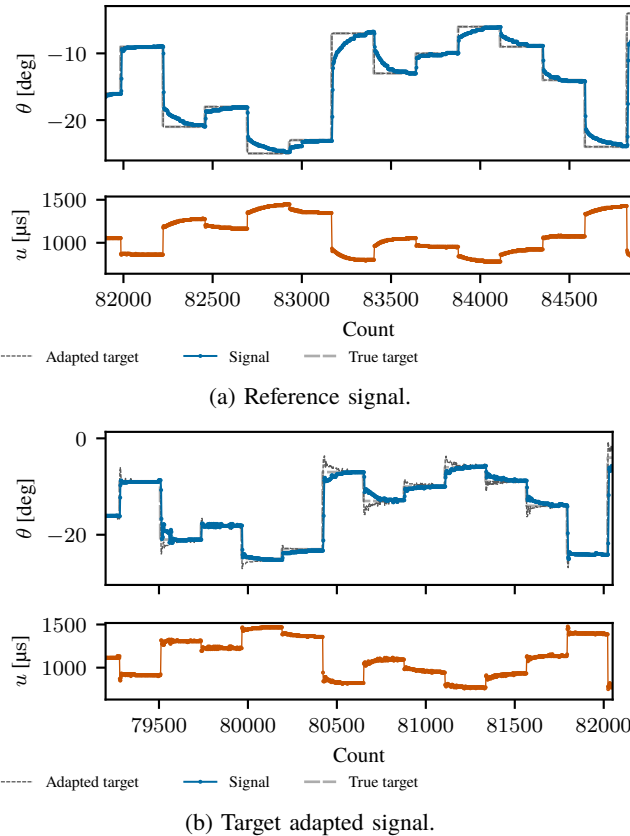


Fig. 14: Qualitative comparison of 1min of the signal response for the *under-tuned* PID controller.



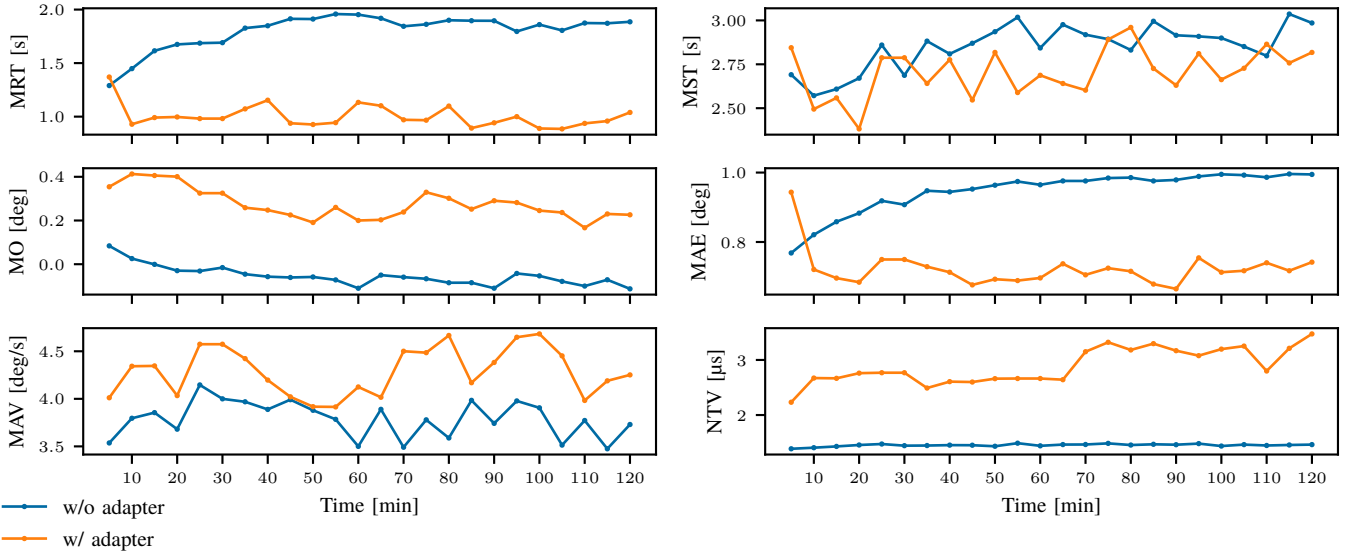


Fig. 15: Metrics evolution over the duration of the experiment for an *under-tuned* PID controller. Each point represents the calculated value of the metric on identical 5min sequences of targets. Numerical values can be found in Appendix C.

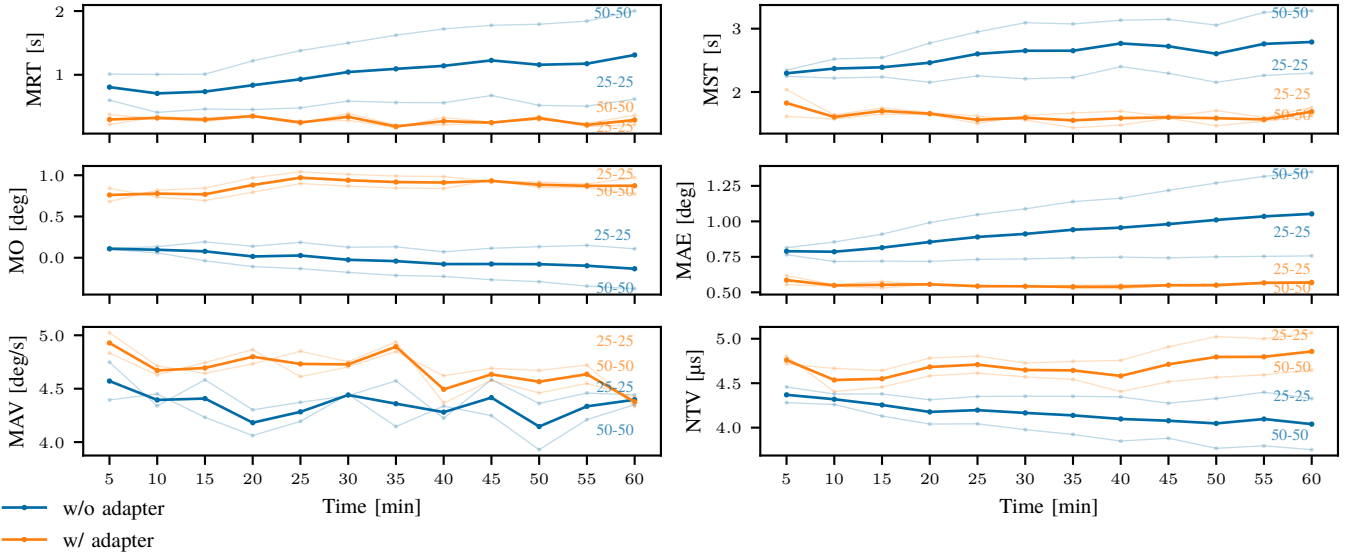


Fig. 16: Metrics evolution over the 1h duration of the experiment for KOSs in a lower state of degradation (0.25–0.25mm and 0.5–0.5mm perforations, see annotations), with their average overlayed. Each point represents the calculated value of the metric on identical 5min sequences of targets. Numerical values can be found in Appendix C.

2) *1.5–0.5 & 2–0.5 Perforations*: The 2–0.5mm perforation aims to resemble a KOS that has been more deteriorated. The PID controller is initially tuned for the 1.5–0.5mm KOS. For the 1.5–0.5mm KOS experiment, with the runtime set to 2h, the aim is to assess how the controllers perform as the system gradually shifts into a more over-tuned condition.

Figure 17 shows that the general performance of the target adapter consists of higher settling times, overshoot, mean absolute velocity and total variance compared to the reference, especially as the experiments progress. In fact, the only metric where the target adapter metrics are smaller than those of the

reference, is the rise time.

Inspecting Figure 18, it can be observed that by the end of the experiment, the target adaptations have become significantly noisy, resulting in chaotic behavior. The result of this is the increased settling time, mean absolute error, mean absolute velocity, and controller effort. However, as shown in Figure 17, the adapted controller initially demonstrates better performance concerning settling time and mean absolute error. Indeed, Figure 18c illustrates that, in the first 30min of the experiment, the target adapter produces minimal and relatively stable adaptations, leading to improved rise times



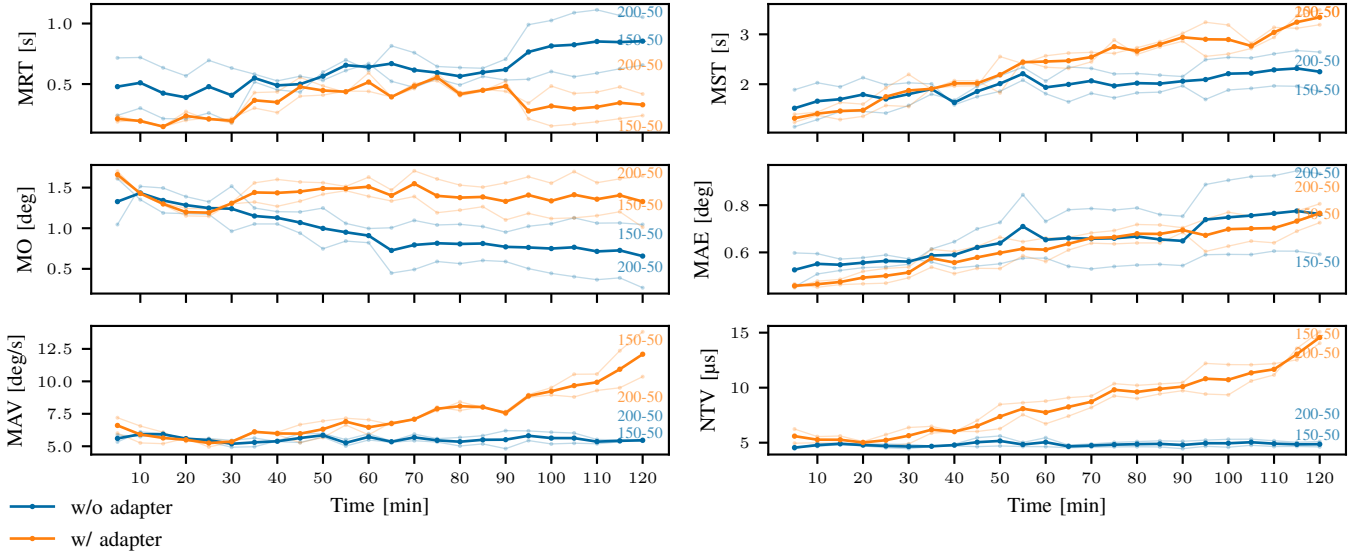


Fig. 17: Metrics evolution over the 2h duration of the experiment for KOSs in a higher state of degradation (1.5–0.5mm and 2–0.5mm perforations, see annotations), with their average overlayed. Each point represents the calculated value of the metric on identical 5min sequences of targets. Numerical values can be found in Appendix C.

and lower absolute error.

*Interpretation:* In the case of heavily deteriorated KOSs, the target adaptation seems to degrade the performance. The initially improved rise times can be attributed to the highly oscillatory nature of the signal response, rather than genuine performance gains. However, the fact that the target adapter shows improvement over the reference controller in the early stages of deterioration is promising. The highly responsive PID controller is quick to react, reducing the need for extreme adaptations as seen in the previous experiments. As the KOS further degrades, it sometimes enters an unstable cycle: much alike the ones seen in Figure 12a, Section VI-A, but at a higher frequency. This results in training data that contains many contradicting features, which results in increased noise in the adapter’s predictions. Noisy targets in turn produce noisy control actions and consequently noisy signal responses.

### C. Crawling Gait

The final experiment emulates the mode of operation that the robotic crawler was designed for. The targets in the sequences alternate between two angles on opposite sides of the potential barrier, specifically  $-8$  and  $-18$  degrees, mimicking the contraction and expansion typical of a crawling motion. To introduce variability and avoid identical datapoints, a small normally distributed random offset is added to each alternating target. The goal of this experiment is to evaluate how a constrained operational setting affects the adapter’s compensation for the overshooting effect caused by the potential barrier. The experiment uses a KOS with a perforation 1.5–0.5mm and the highly responsive PID, tuned to this perforation. Given the rapid degradation of the potential barrier in the early stages, the experiment only runs for 20min. A reference controller is compared with two target adapter configurations:

one pretrained and then fine-tuned online, and one that is entirely trained online.

Figure 19 shows the evolution of the metrics for the three runs over the span of 20min. The results show consistently lower rise and settling times, and mean absolute error for both implementations of the target adapter compared to the reference controller. These improvements come at the cost of significantly higher overshoot.

Signal responses are shown in Figure 20, note that the adapted signal is only shown for the fully online trained adapter, but it is very similar to that of the pretrained adapter. The improvement in rise time is clearly observable, and shows that the improved settling time directly results from this, rather than the reference controller having an unstable signal. In fact, the signal from the reference resembles that of an under-tuned controller. Target adaptations appear to be minimal, being only visual in close-ups of the signal. This corresponds to the relatively low difference in total variance between reference and adapted controllers, observed in the metric evolution.

*Interpretation:* Although the adaptations made to the targets appear negligible for both the pretrained as the online trained adapter, the metrics and signal response show a definite improvement. However, the cause of how these minor adaptations result in the observed improvements is not immediately apparent: the reference controller shows signs of being under-tuned, but the adapter does not produce adaptations similar to those of an under-tuned controller in Section VI-A. On the other hand, the consistent improvements seen across both adapter configurations challenge the notion that these improvements are merely coincidental.

## VII. DISCUSSION

Section VI provides some short interpretation and discussion on the various results presented there.

FOR  
ENTIRE  
SECTION  
quantify t  
results: x'  
improvement of y  
etc.

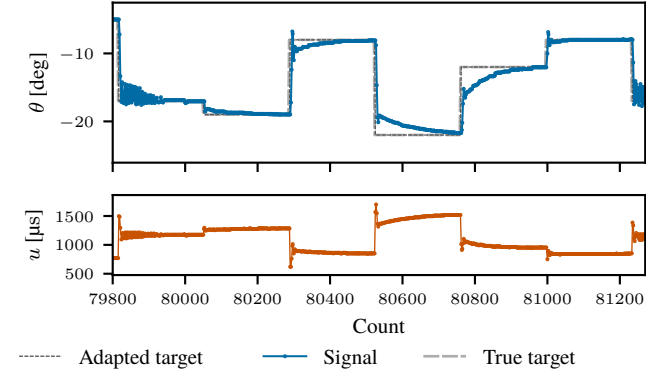
### A. Key Insights

A general trend in the effect of the target adapter can be observed: Whenever a controller-system configuration exhibits under-responsive characteristics, such as the under-tuned controller in Section VI-A and the 0.25–0.25mm & 0.5–0.5mm perforations in Section VI-B, the target adapter induces more aggressive controller behavior. Likewise, when the controller is too aggressive from the start, target adaptations tend to moderate or constrain the behavior.

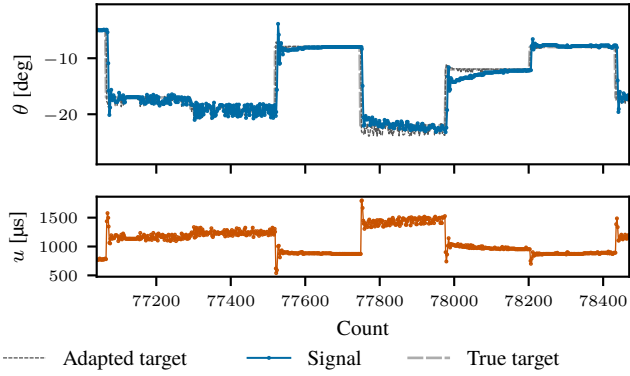
This follows from the interpretation of the evaluation metrics in these scenarios. For example, under-responsive systems exhibit high rise times, and therefore high settling times. These systems are also characterized by low overshoot, low mean absolute velocity, and low controller *effort* (total variance). When the adapter produces more extreme target positions, rise time and settling time decrease, but the overshoot increases. The controller is thus “making an extra effort”, which is observed in the higher total variance: initially setting a more extreme target position, then gradually bringing it closer to the desired target, etc. The overall faster response results in the increased mean absolute velocity. As can be observed in Figure 21, this behavior resembles that of the under-responsive simulation in the toy problem (Section III-C) very closely.

Similarly, systems with an over-aggressive controller, i.e. the *slow-aggressive* controller experiment and the 2–0.5mm & 1.5–0.5mm perforations, exhibit low rise times, but high settling times: the high gains *push* state toward the desired target quickly, and then overcompensate for the high overshoot, resulting in a difficult to maintain a stable response near the target. This explains the high controller *effort*. The adapter moves the target to less extreme positions, compensating for the overshoot “in advance”. This prevents the system from entering an unstable feedback loop, by preventing the control inputs from getting overly aggressive in the first place. Thus, even though target adaptations are made, the overall controller effort is decreased. Figure 22 provides a clearer view of the process. Again, a resemblance to the equivalent scenario in the toy problem can be observed. The contradicting results from the 2–0.5 & 1.5–0.5mm perforations, see Figure 17, will be addressed in Section VII-B. However, as mentioned in Section VI-B, the adapted controller *does* outperform the reference in the early stages, and it is during this phase that the same trends described earlier can be observed.

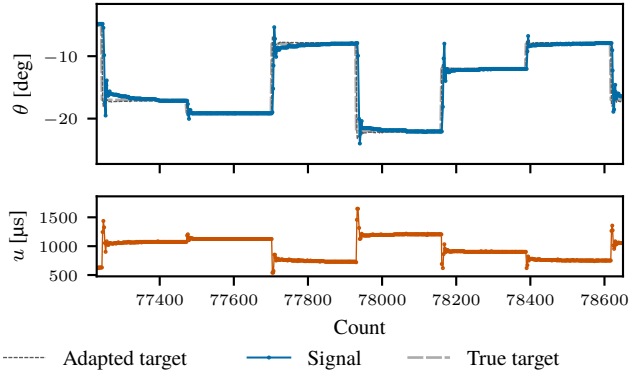
These two observed trends in the target adapter’s effect suggest that the method is inherently self-correcting. By constantly retraining the adapter on features derived from its own influence on the signal response, an under-responsive adapted controller will become more aggressive, up to the point that the adapter needs to moderate its behavior again. This idea is supported by the observation that the evolution of the metrics for adapted controllers tends to stagnate (most clearly visible in Figures 13, 15 and 16). Supposedly, the adapted controller finds a good compromise between moderate and aggressive behavior, and is able to maintain a consistent performance from that point onward.



(a) Reference signal, late in the experiment (last 5min sequence).



(b) Corresponding target adapted signal.



(c) Target adapted signal, early in the experiment (sequence that ends at minute 30).

Fig. 18: Qualitative comparison of 30s of the signal response for the 2–0.5mm perforation. Also showing a comparison between the signals of the target adapted controller, late vs. early into the experiment.

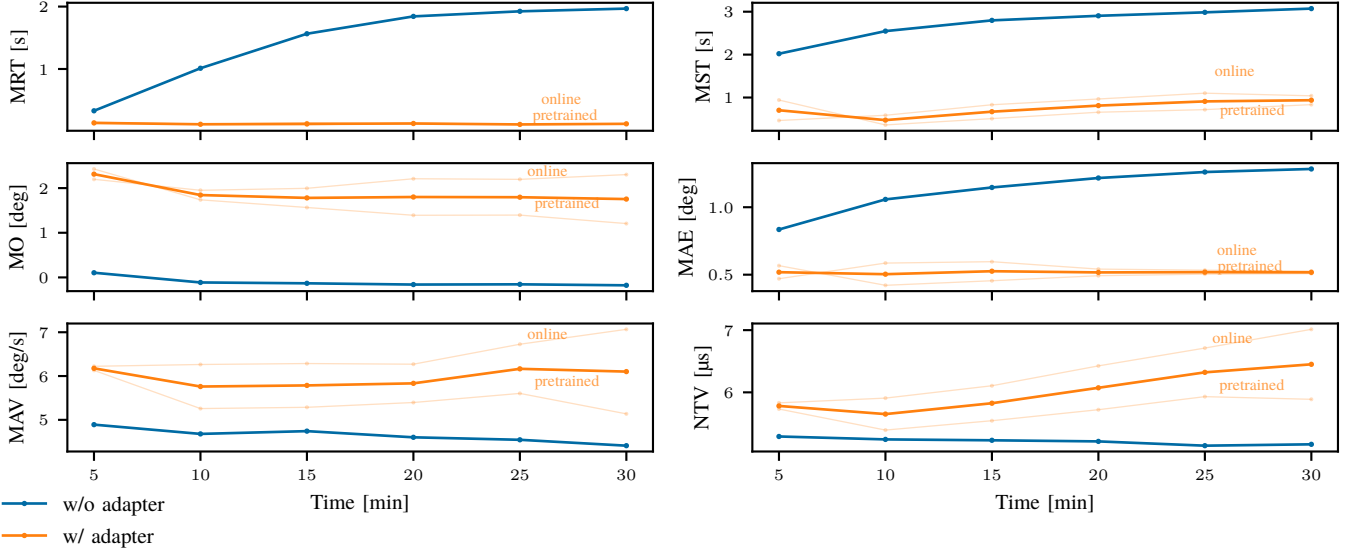


Fig. 19: Metrics evolution over the 20min duration of the crawling-gait experiment, with the average of both adapted controllers overlayed. Each point represents the calculated value of the metric on identical 5min sequences of targets, in this case alternating between values near to -18 and -8 degrees, respectively. Numerical values can be found in Appendix C.

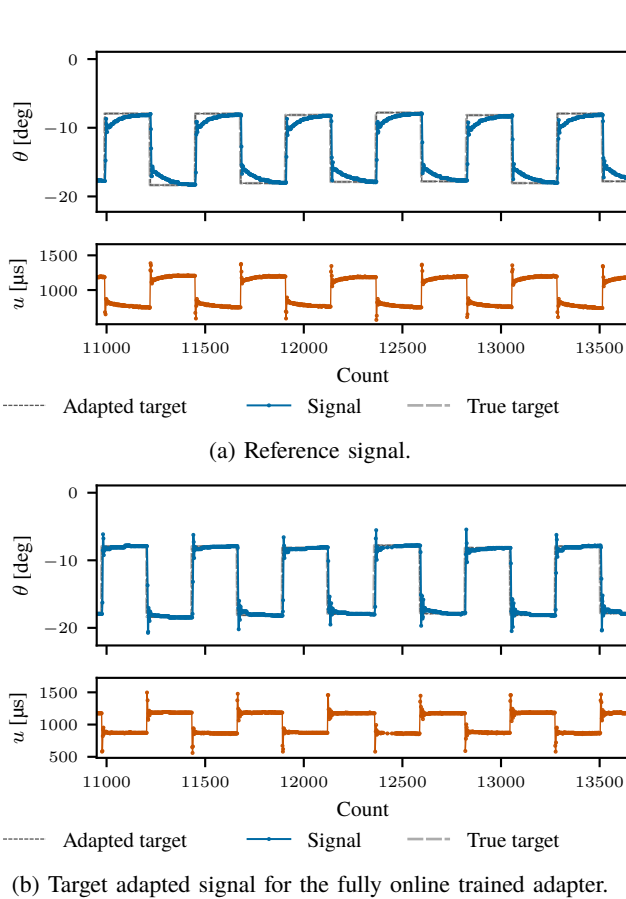


Fig. 20: Qualitative comparison of 1min of the signal response for the 0.5–0.5mm perforation. The snapshot is taken from the first 5min sequence.

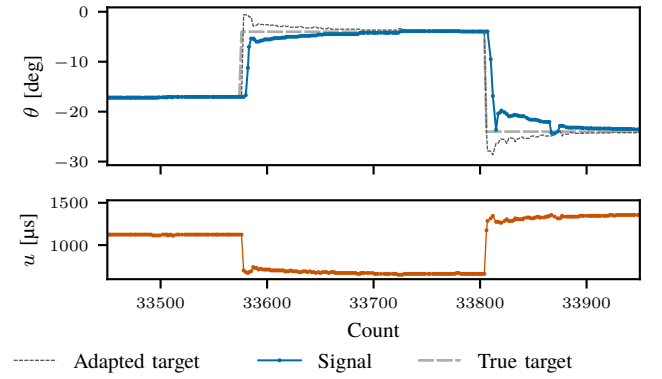


Fig. 21: A close-up from the under-tuned PID controller with target adapter. The signal response generally resembles that of Figure 7 from the toy problem.

### B. Limitations

As discussed in Section VI-B, the experiment using 1.5–0.5 & 2–0.5mm perforations, in combination with a highly responsive PID controller yielded better results for the reference controller than for the adapted controller. The hypothesis is that the training data obtained from the signal response gradually contains more conflicting features, as illustrated by Figure 23. As the controller is highly responsive, the frequency at which the signal response is stored in the buffer is too low to capture sufficient detail.

However, this high responsiveness alone is not solely responsible for the adapter's deteriorating performance, since the adapter initially manages to slightly improve performance. The same reference controller is used in the 0.25–0.25 & 0.5–0.5mm perforation experiments, as well as in the crawling

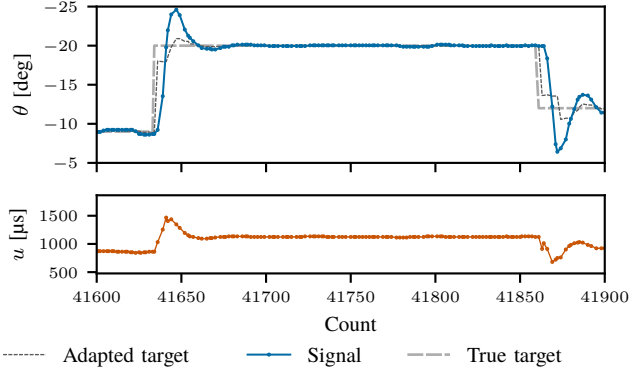


Fig. 22: A close-up from the *slow-aggressive* PID controller with target adapter. The manner in which the target is adapted resembles that of Figure 5 from the toy problem.

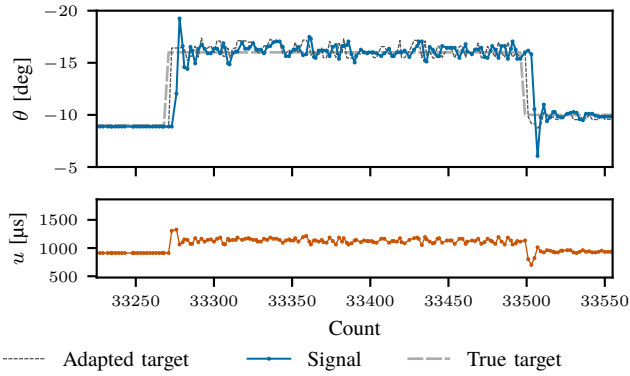


Fig. 23: A close-up of the 2–0.5mm experiment. The oscillations in the signal response are recorded at an insufficient frequency to contain any meaningful relationship between spaced-out state pairs and the target. As a result, the adapter’s output becomes more chaotic as well, further polluting the training data.

gait experiments (Sections VI-B and VI-C, respectively). It seems to be the specific combination of a highly responsive controller—or conversely, a low sampling rate for the buffer—with a more deteriorated KOS that leads to the issue. Once the system enters an unstable feedback loop, the oscillations occur so frequently that any meaningful relationship between sequential datapoints is lost, an observation which becomes clear by comparing Figures 22 and 23. As a result, training features obtained from near-identical regions in the signal response can vary drastically. The adapter thus starts producing less meaningful adaptations, entering a vicious cycle.

This explains why the combination of 1.5–0.5mm perforations and a *slow-aggressive* controller poses no problem, nor does the same perforation combined with the highly responsive controller during the first 30min (also demonstrated by the crawling gait experiment). Similarly, the 0.25–0.25 and 0.5–0.5mm perforations effectively prevent the controller from entering an unstable state.

### C. Validity of the Results

Several important aspects regarding the validity of the results should be addressed.

1) *Reproducibility of Identical KOS*: Since a key part of understanding the results is the use of a reference controller as a baseline, it is important to confirm that the comparisons are representative. Section V already stressed that identical target sequences were used for each comparison. The focus thus shifts to the possible differences in the characteristics of supposedly identical KOSs.

The KOS templates were all designed with identical parameters, exempting the perforations, and were cut out using a laser-cutter. They were, however, assembled by hand. Several steps were taken to mitigate or minimize variability introduced during the assembly.

First, the required KOSs for a certain experiment setting—usually four, since two are required for the reference run, and 2 for the adapter test—were all made in a single *batch*, following a preset routine. Additionally, all KOSs were then “broken in” by exactly one cycle of contracting and expanding it. This procedure reduces potential differences in the cutting and assembly process.

Second, the nature of the robot’s design requires two KOSs per experiment run: as described in Section IV-A, the state of the robot is represented by the angle of the actuation arm. This arm is restrained on both sides by a KOS, each a mirror image of the other. In practice, the combined contribution of both springs averages out, and the influence of any single KOS is minimized.

Finally, during each run of an experiment setting, a “reference minute” was recorded with intervals of 30min. During these reference minutes, and regardless of the specific experiment setting (i.e., with or without the adapter), the robot ran the relevant reference controller on a 1min target sequence. These minutes were evaluated using the same metrics as the experiments to compare the similarity of the KOSs involved. Comparisons of these metrics are presented in Appendix B, but it is sufficient here to note that variations in metric values for reference minutes are insignificant compared to the difference introduced by the implementation of the target adapter.

These measures allow for valid interpretation of the results.

2) *Uninterpreted Results*: For certain experiments, the nature of the adapter’s influence causing the improvement remains unclear. Qualitative results for both adapter configurations of the crawling gait experiment, and the early stage of the 1.5–0.5 & 2–0.5mm perforation experiment (Figures 18 and 20) do not indicate significant adaptations to the target. This might lead to disregarding the results as coincidences. However, the improvement is consistent over all these experiment settings, with notable similarity in the reference signal responses for all settings, as well as adapter signal responses (i.e., signals from seemingly slightly under-responsive systems, and highly responsive systems, respectively).



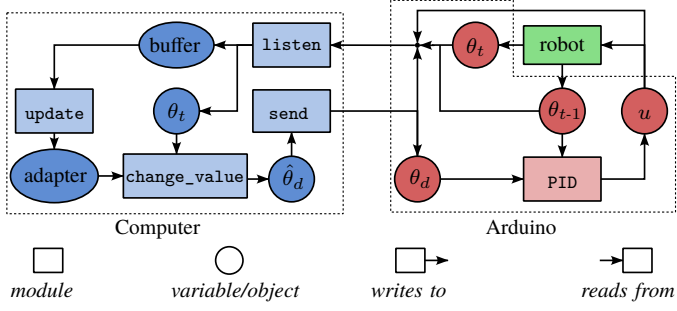


Fig. A.1: The interaction between the modules (rectangles) and shared variables (ovals) of the computer (blue) and Arduino (red).

## VIII. CONCLUSION

### APPENDIX A

#### CODE STRUCTURE

Figure A.1 provides an overview of the communication between computer and Arduino, and their inner interactions between various modules. For example, the sole task of the `send` thread is to read the value of a global variable representing the target state and send this to the Arduino. The `listen` thread reads the tuple sent in return by the Arduino and assigns its contents to the respective shared variables for the state, control action, resulting state and target. It then adds this tuple to the buffer used for updating the adapter model. The `update` thread is responsible for (re)training the adapter. It creates a copy of the adapter, which can then be trained in the background while the original adapter remains functional. Once training is completed, the copied adapter's weights are assigned to the original adapter. The `change_value` thread is responsible for setting the “true” target in the control loop, as well as the actual adaptations to it. All threads aim to operate at 50Hz.

### APPENDIX B

#### COMPARING KOSs

### APPENDIX C

#### EXTENDED TEST RESULTS